Tugas Pertemuan 10.1 – Function & Procedure

Mata Kuliah: Sistem Basis Data (2026)

Prodi: Sarjana Terapan-Teknik Informatika POLBAN

Kelas: 2B

NIM: 251524048

Nama Lengkap: Ghaisan Khoirul Badruzaman

Prasyarat

1. Menggunakan alat bantu atau perangkat lunak RDBMS PostgreSQL
2. Menggunakan skema HR Oracle.

Instruksi

1. Total seluruh soal yang bisa dikerjakan sebanyak **5 soal**.
2. **Disarankan mengerjakan soal secara terturut**!
3. Sertakan jawaban dalam bentuk syntax PL/SQL!
4. Sertakan **screenshot halaman penuh yang menyertakan taskbar dengan waktu** (Print Screen) untuk menunjukkan syntax dan hasil eksekusi query pada RDBMS PostgreSQL!
5. Ekspor file lembar jawaban ini dalam format PDF dengan nama file:
   T10_<Kelas>_<3 NIM Terakhir>.pdf
   contoh sebagai berikut:
   T10_2A_010.pdf
6. Kumpulkan melalui Assignment pada Ms. Teams

Referensi

1. https://neon.com/postgresql/postgresql-plpgsql/introduction-to-postgresql-stored-procedures
2. https://neon.com/postgresql/postgresql-plpgsql/postgresql-create-procedure
3. https://neon.com/postgresql/postgresql-plpgsql/postgresql-drop-function

Jika ada hal yang ingin ditanyakan, silahkan sampaikan melalui WhatsApp Group/Ms. Teams.

Selamat mengerjakan!

Soal No. 1

Soal

a. Buatlah function "salary_grade" untuk menentukan grade salary berdasarkan ketentuan berikut:
   - S: mulai dari 13.000
   - A: mulai dari 10.000
   - B: mulai dari 7.000
   - C: mulai dari 5.000
   - D: di bawah 5.000

b. Panggil function menggunakan query sebagai berikut:
   SELECT employee_id, first_name, last_name, salary, salary_grade(salary) FROM employees;

Jawaban (Penjelasan dan Syntax)

Persiapan: masuk ke database hr (schema HR Oracle untuk PostgreSQL dari modb.pro/db/100650)

Masuk ke psql sebagai user postgres, mematikan pager supaya hasil query yang panjang tidak berhenti di tengah, lalu pindah ke database hr. Tabel employees berisi 107 baris.

```
sudo -u postgres psql

\pset pager off

\c hr

SELECT COUNT(*) AS jumlah_employee FROM employees;
```

Soal 1: membuat function salary_grade

Function salary_grade menerima satu parameter p_salary bertipe NUMERIC (sama dengan tipe kolom salary, numeric(8,2)) dan mengembalikan CHAR(1) berisi huruf grade.

Pengecekan memakai IF-ELSIF dari batas paling tinggi ke paling rendah. Karena urutannya dari atas, setiap cabang cukup memeriksa batas bawahnya saja. Contohnya salary 12000 tidak lolos >= 13000 tapi lolos >= 10000, jadi grade A. Salary di bawah 5000 jatuh ke ELSE dan mendapat grade D.

Kalau salary NULL, function mengembalikan NULL supaya employee tanpa data gaji tidak ikut dianggap grade D.

\df salary_grade menunjukkan function sudah tersimpan dengan tipe func, parameter p_salary numeric, dan tipe hasil character.

```
CREATE OR REPLACE FUNCTION salary_grade(p_salary NUMERIC)
  RETURNS CHAR(1)
  LANGUAGE plpgsql
AS $$
BEGIN
  IF p_salary IS NULL THEN
    RETURN NULL;
  ELSIF p_salary >= 13000 THEN
    RETURN 'S';
  ELSIF p_salary >= 10000 THEN
    RETURN 'A';
  ELSIF p_salary >= 7000 THEN
    RETURN 'B';
  ELSIF p_salary >= 5000 THEN
    RETURN 'C';
  ELSE
    RETURN 'D';
  END IF;
END;
$$;

\df salary_grade
```

Soal 1: memanggil function salary_grade dengan query dari soal

Function dipanggil di dalam SELECT seperti fungsi bawaan, sekali untuk setiap baris employees. Hasilnya 107 baris dengan kolom tambahan salary_grade.

Contoh hasil: Steven King (24000) grade S, Nancy Greenberg (12000) grade A, Alexander Hunold (9000) grade B, Bruce Ernst (6000) grade C, dan David Austin (4800) grade D.

Output 107 baris lebih panjang dari layar, jadi screenshot pertama diambil dengan layar digulung ke atas supaya query terlihat, dan screenshot kedua memperlihatkan bagian akhir hasil.

```
SELECT employee_id, first_name, last_name, salary, salary_grade(salary) FROM
    employees;
```

Soal 1 (percobaan tambahan): nilai batas tiap grade dan jumlah employee per grade

Percobaan tambahan untuk memastikan batas tiap grade benar. Nilai tepat di batas (13000, 10000, 7000, 5000) masuk grade yang lebih tinggi karena syaratnya "mulai dari" (>=), sedangkan nilai 0,01 di bawahnya turun satu grade.

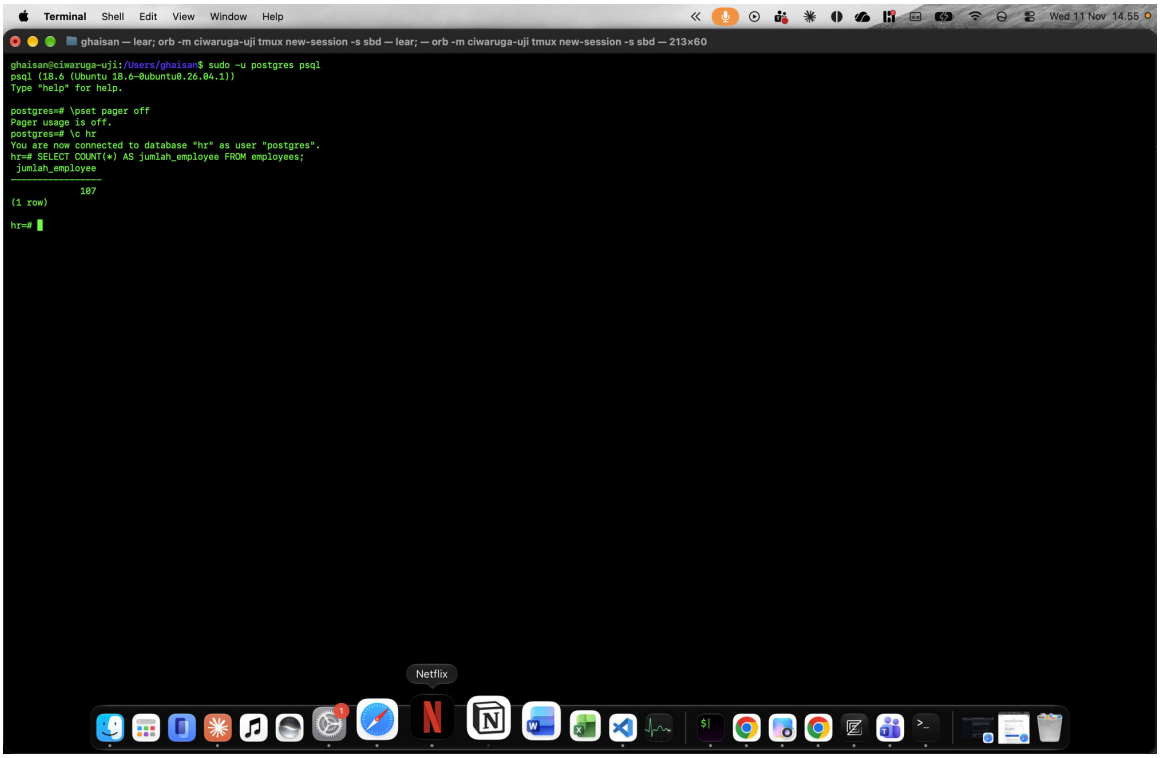
Query kedua mengelompokkan employee berdasarkan hasil function: grade S 6 orang (13000 sampai 24000), A 13 orang, B 28 orang, C 11 orang, dan D 49 orang (2100 sampai 4800). Totalnya 107, jadi tidak ada employee yang terlewat. Hasil function juga bisa dipakai di GROUP BY.

```
SELECT s AS salary, salary_grade(s) AS grade
FROM (VALUES (13000), (12999.99), (10000), (9999.99), (7000), (6999.99), (5000),
    (4999.99)) AS v(s);

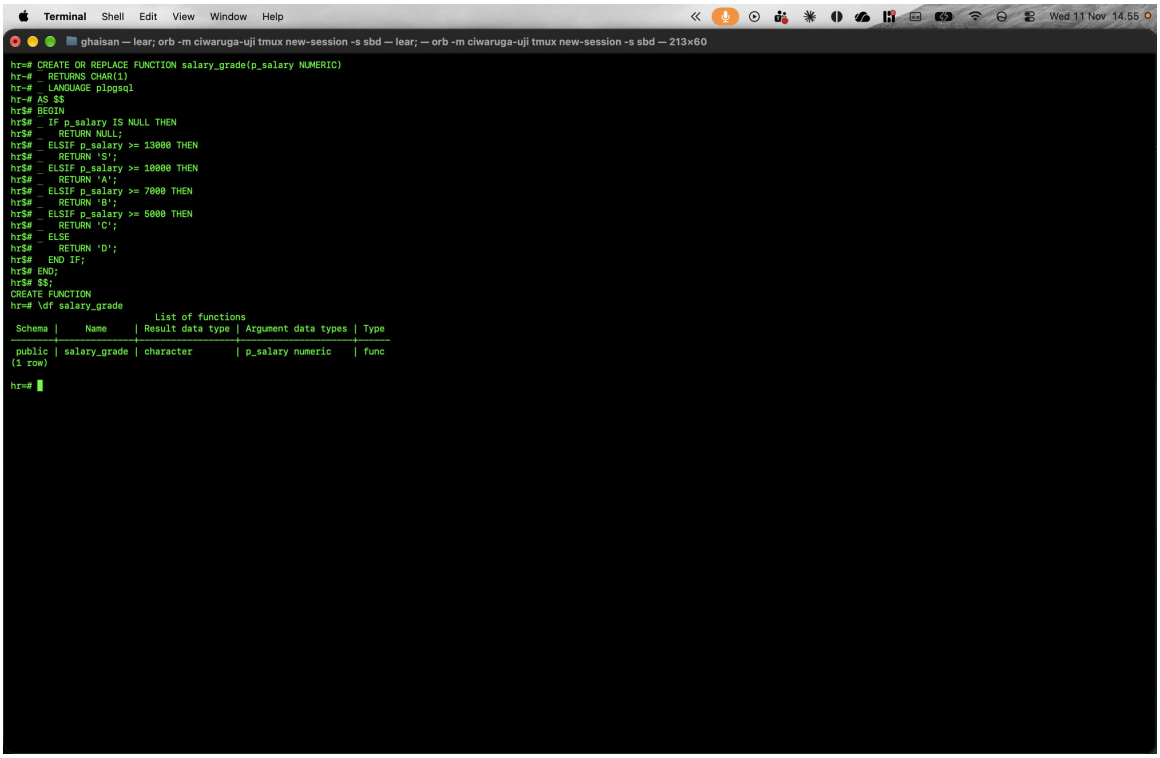
SELECT salary_grade(salary) AS grade, COUNT(*) AS jumlah, MIN(salary), MAX(salary)
FROM employees
GROUP BY salary_grade(salary)
ORDER BY MIN(salary) DESC;
```

Hasil Eksekusi (Screenshot)

Persiapan: masuk ke database hr (schema HR Oracle untuk PostgreSQL dari modb.pro/db/100650)



Soal 1: membuat function salary_grade

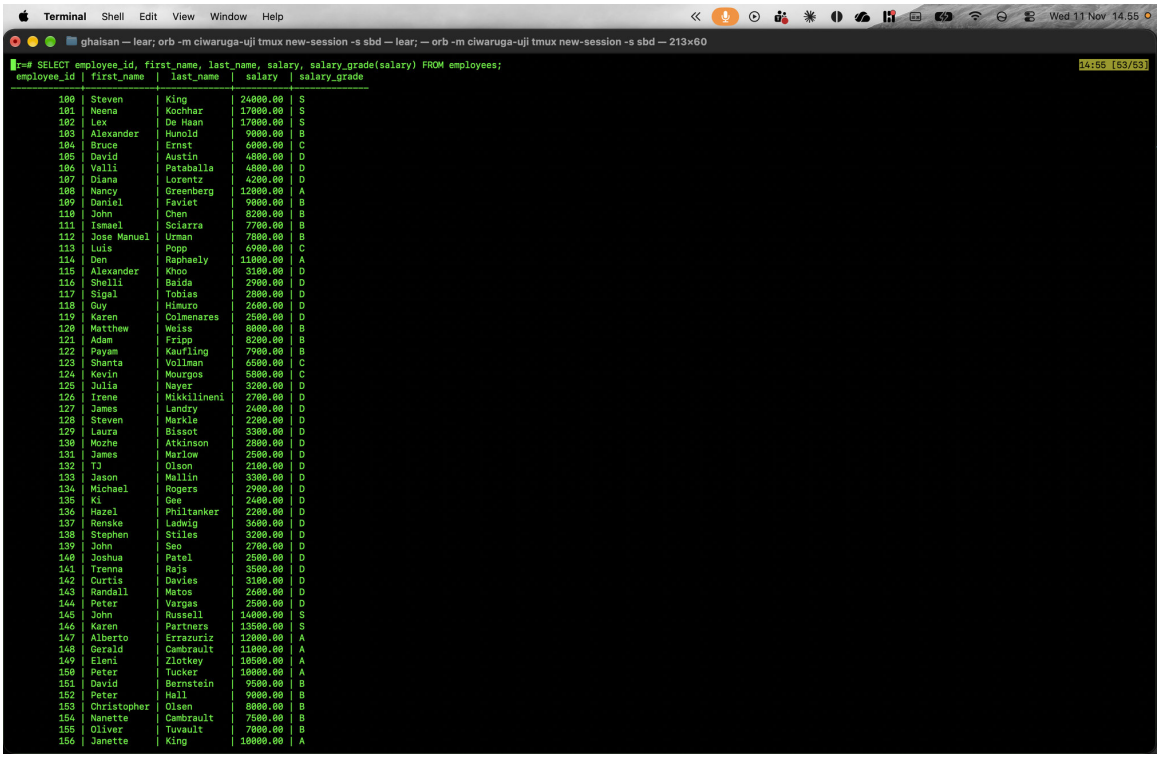


```
Terminal  Shell  Edit  View  Window  Help                                   Wed 11 Nov 14.55
ghaisan — lear; orb -m ciwaruga-uji tmux new-session -s sbd — lear; — orb -m ciwaruga-uji tmux new-session -s sbd — 213×60

hr=# CREATE OR REPLACE FUNCTION salary_grade(p_salary NUMERIC)
hr-# RETURNS CHAR(1)
hr-# LANGUAGE plpgsql
hr-# AS $$
hr$# BEGIN
hr$# IF p_salary IS NULL THEN
hr$#   RETURN NULL;
hr$# ELSIF p_salary >= 13000 THEN
hr$#   RETURN 'S';
hr$# ELSIF p_salary >= 10000 THEN
hr$#   RETURN 'A';
hr$# ELSIF p_salary >= 7000 THEN
hr$#   RETURN 'B';
hr$# ELSIF p_salary >= 5000 THEN
hr$#   RETURN 'C';
hr$# ELSE
hr$#   RETURN 'D';
hr$# END IF;
hr$# END;
hr$# $$;
CREATE FUNCTION
hr=# \df salary_grade
                                  List of functions
 Schema |     Name     | Result data type | Argument data types | Type
--------+--------------+------------------+---------------------+------
 public | salary_grade | character        | p_salary numeric    | func
(1 row)

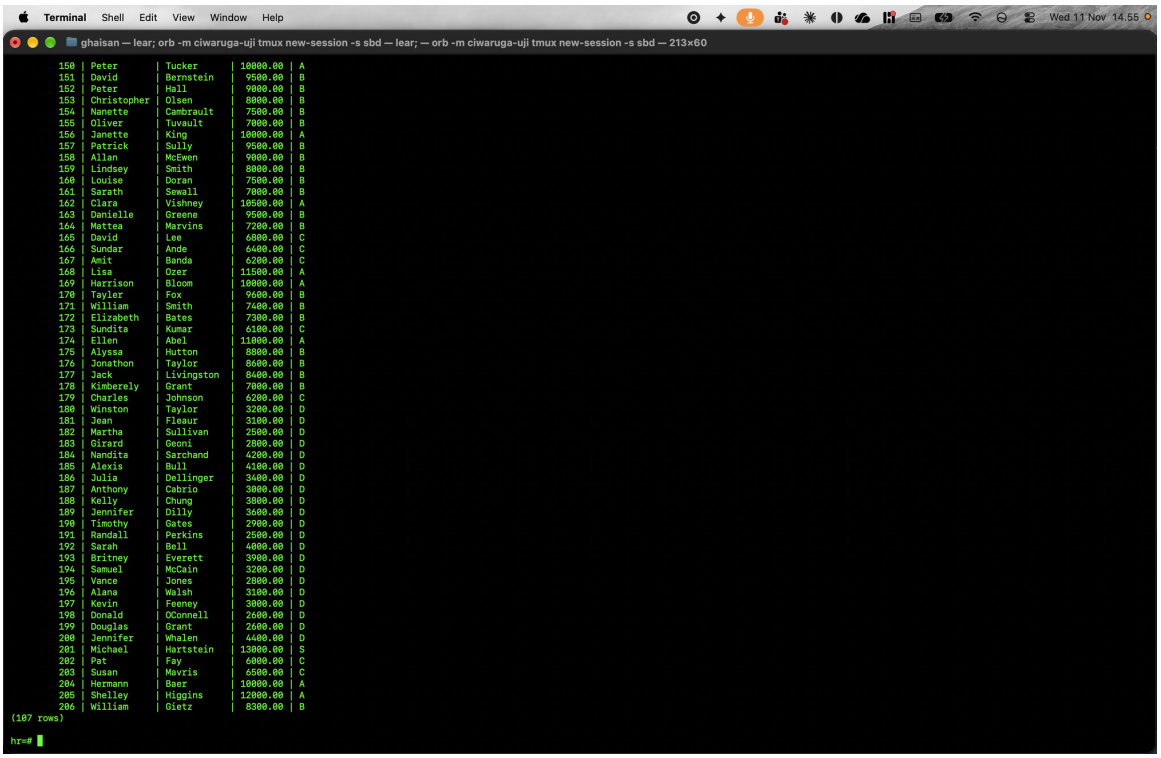
hr=#
```

Soal 1: memanggil function salary_grade dengan query dari soal

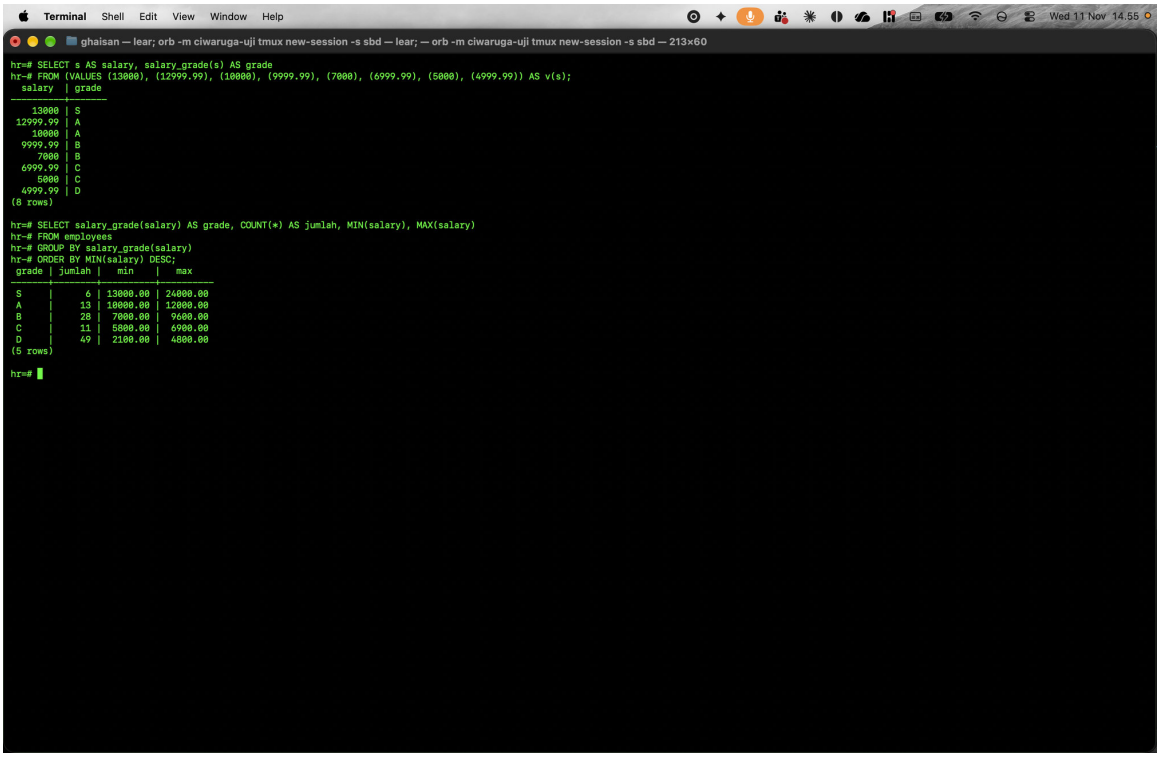


```
Terminal  Shell  Edit  View  Window  Help                                   Wed 11 Nov 14.55
ghaisan — lear; orb -m ciwaruga-uji tmux new-session -s sbd — lear; — orb -m ciwaruga-uji tmux new-session -s sbd — 213×60

hr=# SELECT employee_id, first_name, last_name, salary, salary_grade(salary) FROM employees;                    14:55 [53/53]
 employee_id | first_name  |  last_name  |  salary  | salary_grade
-------------+-------------+-------------+----------+--------------
         100 | Steven      | King        | 24000.00 | S
         101 | Neena       | Kochhar     | 17000.00 | S
         102 | Lex         | De Haan     | 17000.00 | S
         103 | Alexander   | Hunold      |  9000.00 | B
         104 | Bruce       | Ernst       |  6000.00 | C
         105 | David       | Austin      |  4800.00 | D
         106 | Valli       | Pataballa   |  4800.00 | D
         107 | Diana       | Lorentz     |  4200.00 | D
         108 | Nancy       | Greenberg   | 12000.00 | A
         109 | Daniel      | Faviet      |  9000.00 | B
         110 | John        | Chen        |  8200.00 | B
         111 | Ismael      | Sciarra     |  7700.00 | B
         112 | Jose Manuel | Urman       |  7800.00 | B
         113 | Luis        | Popp        |  6900.00 | C
         114 | Den         | Raphaely    | 11000.00 | A
         115 | Alexander   | Khoo        |  3100.00 | D
         116 | Shelli      | Baida       |  2900.00 | D
         117 | Sigal       | Tobias      |  2800.00 | D
         118 | Guy         | Himuro      |  2600.00 | D
         119 | Karen       | Colmenares  |  2500.00 | D
         120 | Matthew     | Weiss       |  8000.00 | B
         121 | Adam        | Fripp       |  8200.00 | B
         122 | Payam       | Kaufling    |  7900.00 | B
         123 | Shanta      | Vollman     |  6500.00 | C
         124 | Kevin       | Mourgos     |  5800.00 | C
         125 | Julia       | Nayer       |  3200.00 | D
         126 | Irene       | Mikkilineni |  2700.00 | D
         127 | James       | Landry      |  2400.00 | D
         128 | Steven      | Markle      |  2200.00 | D
         129 | Laura       | Bissot      |  3300.00 | D
         130 | Mozhe       | Atkinson    |  2800.00 | D
         131 | James       | Marlow      |  2500.00 | D
         132 | TJ          | Olson       |  2100.00 | D
         133 | Jason       | Mallin      |  3300.00 | D
         134 | Michael     | Rogers      |  2900.00 | D
         135 | Ki          | Gee         |  2400.00 | D
         136 | Hazel       | Philtanker  |  2200.00 | D
         137 | Renske      | Ladwig      |  3600.00 | D
         138 | Stephen     | Stiles      |  3200.00 | D
         139 | John        | Seo         |  2700.00 | D
         140 | Joshua      | Patel       |  2500.00 | D
         141 | Trenna      | Rajs        |  3500.00 | D
         142 | Curtis      | Davies      |  3100.00 | D
         143 | Randall     | Matos       |  2600.00 | D
         144 | Peter       | Vargas      |  2500.00 | D
         145 | John        | Russell     | 14000.00 | S
         146 | Karen       | Partners    | 13500.00 | S
         147 | Alberto     | Errazuriz   | 12000.00 | A
         148 | Gerald      | Cambrault   | 11000.00 | A
         149 | Eleni       | Zlotkey     | 10500.00 | A
         150 | Peter       | Tucker      | 10000.00 | A
         151 | David       | Bernstein   |  9500.00 | B
         152 | Peter       | Hall        |  9000.00 | B
         153 | Christopher | Olsen       |  8000.00 | B
         154 | Nanette     | Cambrault   |  7500.00 | B
         155 | Oliver      | Tuvault     |  7000.00 | B
         156 | Janette     | King        | 10000.00 | A
```



Soal 1 (percobaan tambahan): nilai batas tiap grade dan jumlah employee per grade



Soal No. 2

Soal

a. Buatlah procedure "update_suffix_email" untuk memperbarui data email sehingga disetiap baris nilai email ditambahkan "@ORACLE.COM" dibelakangnya!

b. Lakukan query berikut untuk memeriksa keberhasilkan procedure tersebut:
SELECT employee_id, first_name, last_name, email FROM employees;

c. Buatlah procedure "restore_suffix_email" untuk mengembalikan data email kembali ke semula dengan menghilangkan suffix "@ORACLE.COM"!

d. Lakukan query berikut untuk memeriksa keberhasilkan procedure tersebut:
SELECT employee_id, first_name, last_name, email FROM employees;

Jawaban (Penjelasan dan Syntax)

Soal 2: membuat dan memanggil procedure update_suffix_email

Procedure dibuat dengan CREATE PROCEDURE dan tidak punya RETURNS karena tugasnya menjalankan aksi (mengubah data), bukan mengembalikan nilai.
Isinya satu UPDATE yang menyambung email dengan '@ORACLE.COM' memakai operator ||. Kondisi WHERE email NOT LIKE '%@ORACLE.COM' menjaga supaya email yang sudah punya suffix tidak ditambah lagi kalau procedure dipanggil dua kali.
GET DIAGNOSTICS jumlah = ROW_COUNT mengambil jumlah baris yang terkena UPDATE terakhir, lalu ditampilkan dengan RAISE NOTICE.
Procedure dijalankan dengan CALL update_suffix_email(); dan muncul NOTICE 107 email ditambah suffix @ORACLE.COM. Hasil SELECT menunjukkan semua email berubah, misalnya SKING menjadi SKING@ORACLE.COM. Email terpanjang 8 karakter, ditambah 11 karakter suffix jadi 19, masih muat di kolom email VARCHAR(25).

```
CREATE OR REPLACE PROCEDURE update_suffix_email()
  LANGUAGE plpgsql
AS $$
DECLARE
  jumlah INTEGER;
BEGIN
  UPDATE employees
  SET email = email || '@ORACLE.COM'
  WHERE email NOT LIKE '%@ORACLE.COM';

  GET DIAGNOSTICS jumlah = ROW_COUNT;
  RAISE NOTICE '% email ditambah suffix @ORACLE.COM', jumlah;
END;
$$;

CALL update_suffix_email();

SELECT employee_id, first_name, last_name, email FROM employees;
```

Soal 2: membuat dan memanggil procedure restore_suffix_email

restore_suffix_email membuang 11 karakter terakhir (panjang '@ORACLE.COM' adalah 11) dengan LEFT(email, LENGTH(email) - LENGTH('@ORACLE.COM')). Yang diproses hanya email yang memang berakhiran @ORACLE.COM (WHERE email LIKE '%@ORACLE.COM'), jadi email lain tidak ikut terpotong.
Setelah CALL restore_suffix_email(); muncul NOTICE 107 email dikembalikan ke semula, dan hasil SELECT menunjukkan email kembali seperti awal, misalnya SKING@ORACLE.COM kembali menjadi SKING.

```
CREATE OR REPLACE PROCEDURE restore_suffix_email()
  LANGUAGE plpgsql
AS $$
DECLARE
  jumlah INTEGER;
```

```
BEGIN
  UPDATE employees
  SET email = LEFT(email, LENGTH(email) - LENGTH('@ORACLE.COM'))
  WHERE email LIKE '%@ORACLE.COM';

  GET DIAGNOSTICS jumlah = ROW_COUNT;
  RAISE NOTICE '% email dikembalikan ke semula', jumlah;
END;
$$;

CALL restore_suffix_email();

SELECT employee_id, first_name, last_name, email FROM employees;
```

Soal 2 (percobaan tambahan): procedure dipanggil dua kali berturut-turut

Percobaan tambahan untuk menguji kondisi WHERE di kedua procedure. update_suffix_email dipanggil dua kali: panggilan pertama mengubah 107 email, panggilan kedua 0 email karena semua email sudah bersuffix. Hasil pengecekan 107 dari 107 email bersuffix, tidak ada yang menjadi @ORACLE.COM@ORACLE.COM.

Hal yang sama untuk restore_suffix_email: panggilan kedua mengubah 0 baris, dan di akhir tidak ada lagi email bersuffix (0 dari 107). Data email kembali seperti semula untuk soal berikutnya.

```
CALL update_suffix_email();

CALL update_suffix_email();

SELECT COUNT(*) FILTER (WHERE email LIKE '%@ORACLE.COM') AS ada_suffix, COUNT(*) AS
    total FROM employees;

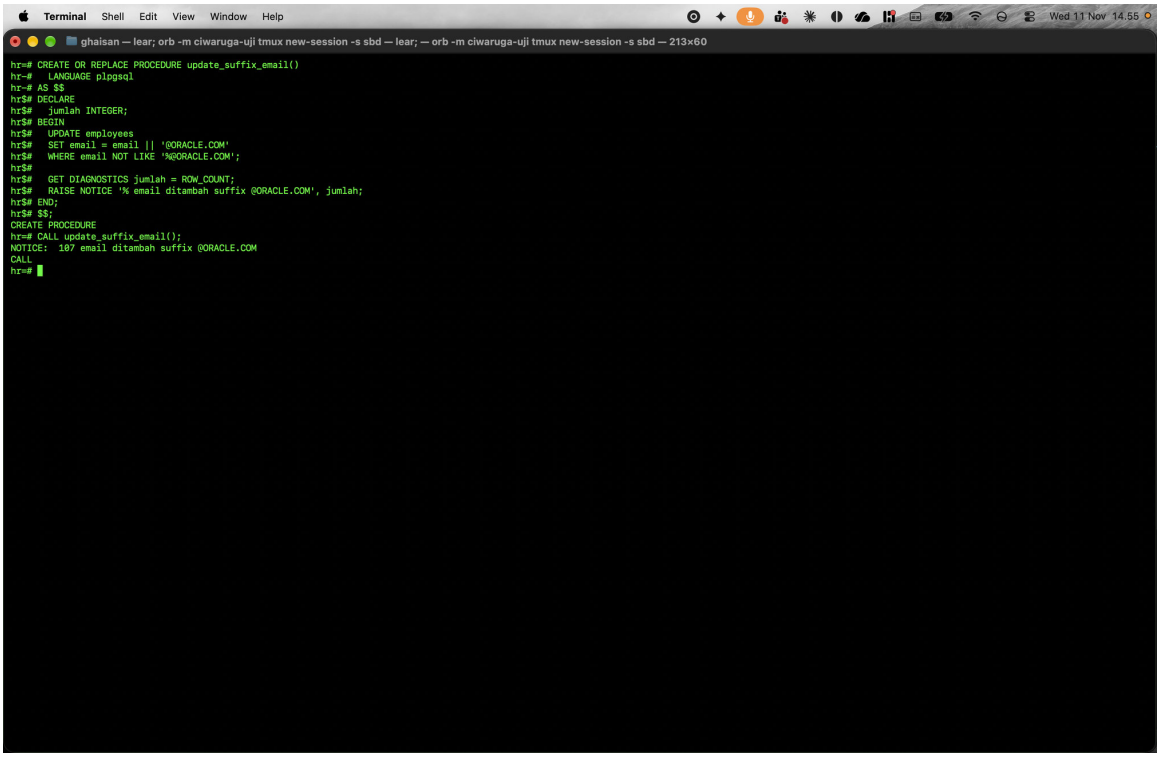
CALL restore_suffix_email();

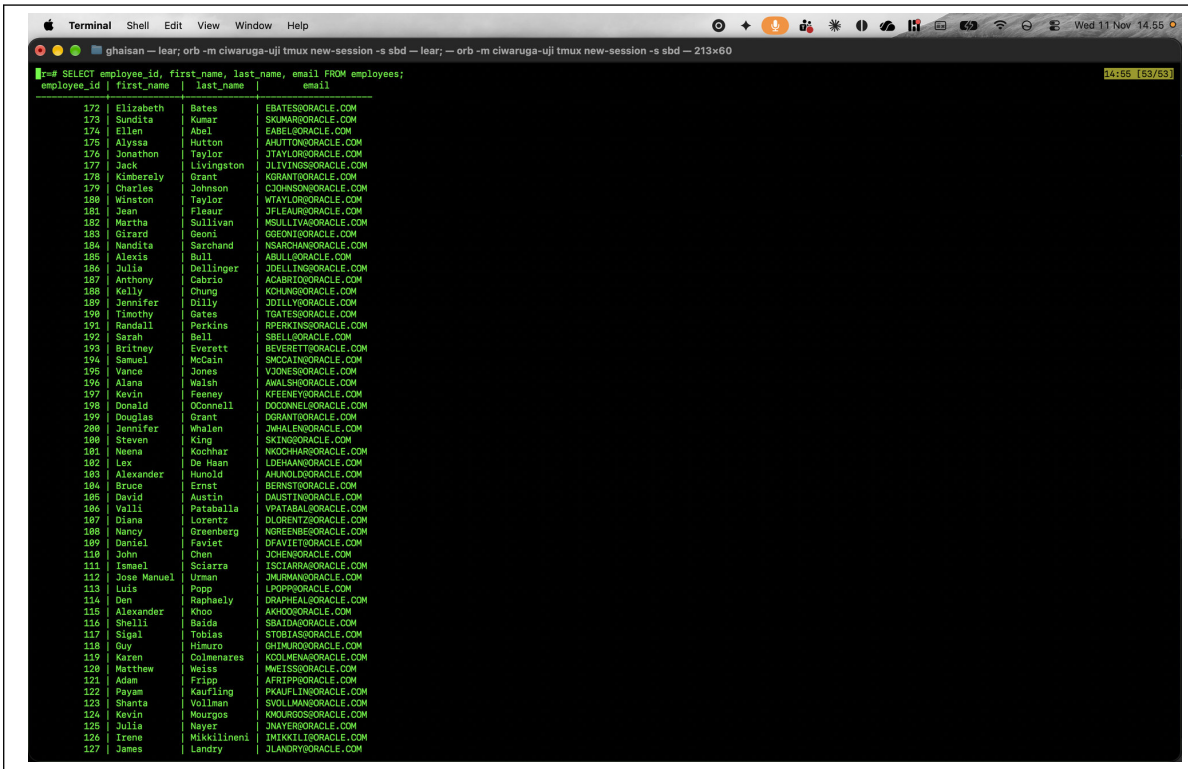
CALL restore_suffix_email();

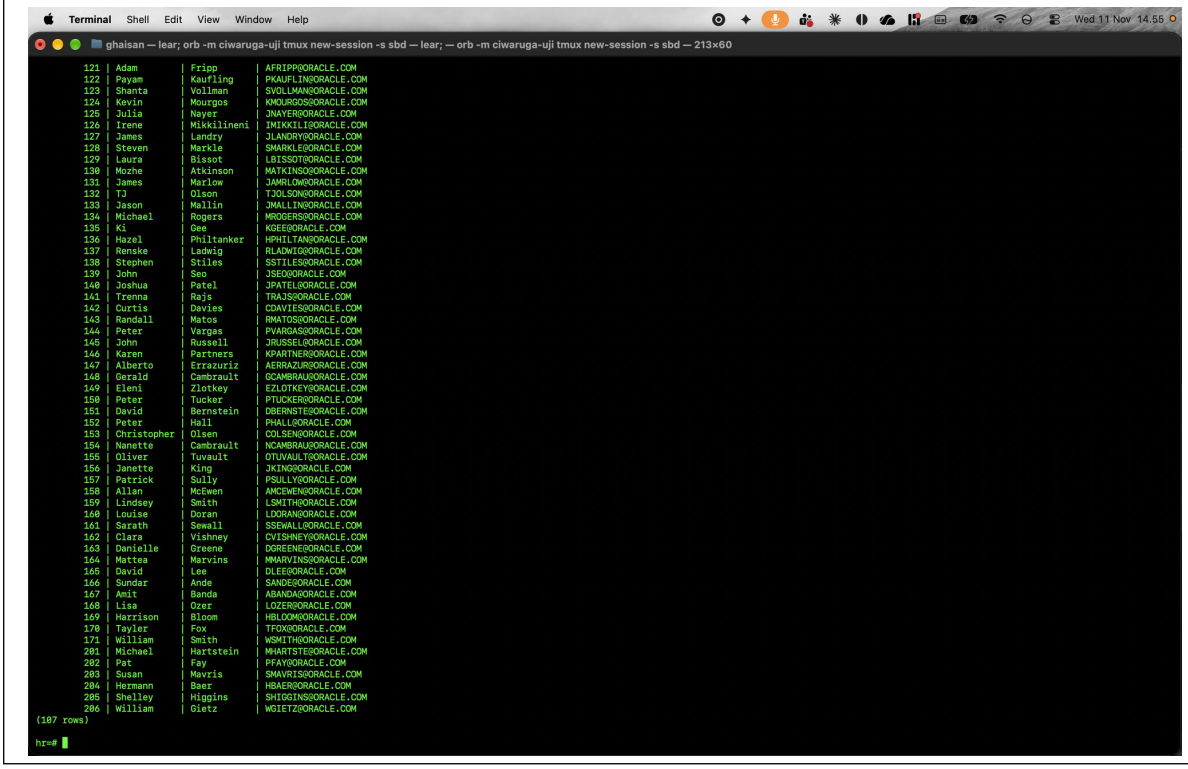
SELECT COUNT(*) FILTER (WHERE email LIKE '%@ORACLE.COM') AS ada_suffix, COUNT(*) AS
    total FROM employees;
```

Hasil Eksekusi (Screenshot)

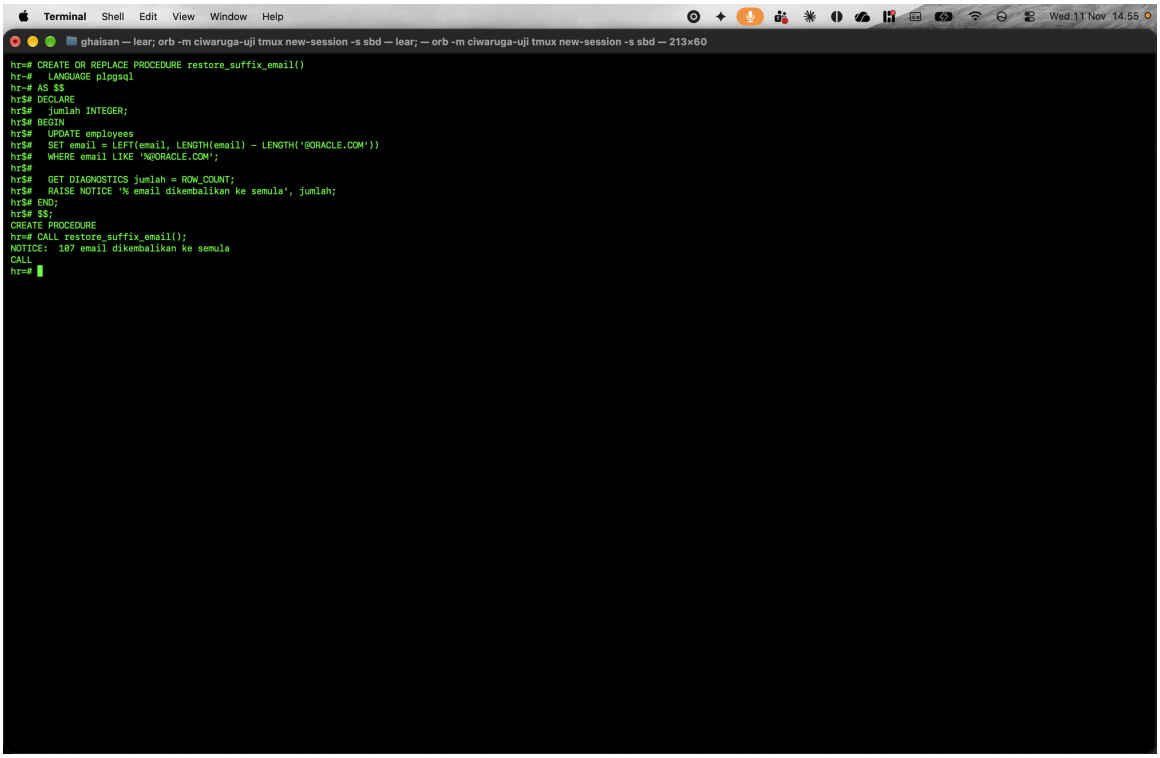
Soal 2: membuat dan memanggil procedure update_suffix_email




Terminal Shell Edit View Window Help
Wed 11 Nov 14.55
ghaisan — lear; orb -m ciwaruga-uji tmux new-session -s sbd — lear; — orb -m ciwaruga-uji tmux new-session -s sbd — 213×60
hr# SELECT employee_id, first_name, last_name, email FROM employees;
14:55 [53/53]
employee_id | first_name | last_name | email
172 | Elizabeth | Bates | EBATES@ORACLE.COM
173 | Sundita | Kumar | SKUMAR@ORACLE.COM
174 | Ellen | Abel | EABEL@ORACLE.COM
175 | Alyssa | Hutton | AHUTTON@ORACLE.COM
176 | Jonathon | Taylor | JTAYLOR@ORACLE.COM
177 | Jack | Livingston | JLIVINGS@ORACLE.COM
178 | Kimberely | Grant | KGRANT@ORACLE.COM
179 | Charles | Johnson | CJOHNSON@ORACLE.COM
180 | Winston | Taylor | WTAYLOR@ORACLE.COM
181 | Jean | Fleaur | JFLEAUR@ORACLE.COM
182 | Martha | Sullivan | MSULLIVA@ORACLE.COM
183 | Girard | Geoni | GGEONI@ORACLE.COM
184 | Nandita | Sarchand | NSARCHAN@ORACLE.COM
185 | Alexis | Bull | ABULL@ORACLE.COM
186 | Julia | Dellinger | JDELLING@ORACLE.COM
187 | Anthony | Cabrio | ACABRIO@ORACLE.COM
188 | Kelly | Chung | KCHUNG@ORACLE.COM
189 | Jennifer | Dilly | JDILLY@ORACLE.COM
190 | Timothy | Gates | TGATES@ORACLE.COM
191 | Randall | Perkins | RPERKINS@ORACLE.COM
192 | Sarah | Bell | SBELL@ORACLE.COM
193 | Britney | Everett | BEVERETT@ORACLE.COM
194 | Samuel | McCain | SMCCAIN@ORACLE.COM
195 | Vance | Jones | VJONES@ORACLE.COM
196 | Alana | Walsh | AWALSH@ORACLE.COM
197 | Kevin | Feeney | KFEENEY@ORACLE.COM
198 | Donald | OConnell | DOCONNEL@ORACLE.COM
199 | Douglas | Grant | DGRANT@ORACLE.COM
200 | Jennifer | Whalen | JWHALEN@ORACLE.COM
100 | Steven | King | SKING@ORACLE.COM
101 | Neena | Kochhar | NKOCHHAR@ORACLE.COM
102 | Lex | De Haan | LDEHAAN@ORACLE.COM
103 | Alexander | Hunold | AHUNOLD@ORACLE.COM
104 | Bruce | Ernst | BERNST@ORACLE.COM
105 | David | Austin | DAUSTIN@ORACLE.COM
106 | Valli | Pataballa | VPATABAL@ORACLE.COM
107 | Diana | Lorentz | DLORENTZ@ORACLE.COM
108 | Nancy | Greenberg | NGREENBE@ORACLE.COM
109 | Daniel | Faviet | DFAVIET@ORACLE.COM
110 | John | Chen | JCHEN@ORACLE.COM
111 | Ismael | Sciarra | ISCIARRA@ORACLE.COM
112 | Jose Manuel | Urman | JMURMAN@ORACLE.COM
113 | Luis | Popp | LPOPP@ORACLE.COM
114 | Den | Raphaely | DRAPHEAL@ORACLE.COM
115 | Alexander | Khoo | AKHOO@ORACLE.COM
116 | Shelli | Baida | SBAIDA@ORACLE.COM
117 | Sigal | Tobias | STOBIAS@ORACLE.COM
118 | Guy | Himuro | GHIMURO@ORACLE.COM
119 | Karen | Colmenares | KCOLMENA@ORACLE.COM
120 | Matthew | Weiss | MWEISS@ORACLE.COM
121 | Adam | Fripp | AFRIPP@ORACLE.COM
122 | Payam | Kaufling | PKAUFLIN@ORACLE.COM
123 | Shanta | Vollman | SVOLLMAN@ORACLE.COM
124 | Kevin | Mourgos | KMOURGOS@ORACLE.COM
125 | Julia | Nayer | JNAYER@ORACLE.COM
126 | Irene | Mikkilineni | IMIKKILI@ORACLE.COM
127 | James | Landry | JLANDRY@ORACLE.COM

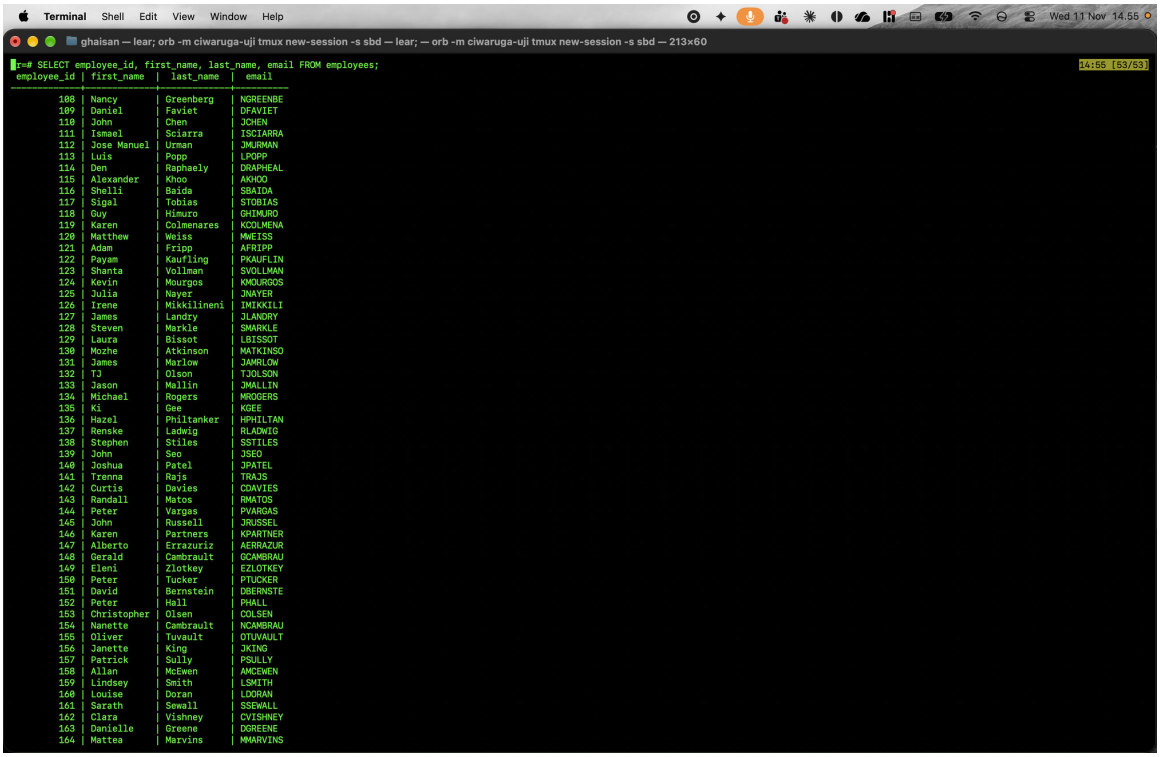

Terminal Shell Edit View Window Help
Wed 11 Nov 14.55
ghaisan — lear; orb -m ciwaruga-uji tmux new-session -s sbd — lear; — orb -m ciwaruga-uji tmux new-session -s sbd — 213×60
121 | Adam | Fripp | AFRIPP@ORACLE.COM
122 | Payam | Kaufling | PKAUFLIN@ORACLE.COM
123 | Shanta | Vollman | SVOLLMAN@ORACLE.COM
124 | Kevin | Mourgos | KMOURGOS@ORACLE.COM
125 | Julia | Nayer | JNAYER@ORACLE.COM
126 | Irene | Mikkilineni | IMIKKILI@ORACLE.COM
127 | James | Landry | JLANDRY@ORACLE.COM
128 | Steven | Markle | SMARKLE@ORACLE.COM
129 | Laura | Bissot | LBISSOT@ORACLE.COM
130 | Mozhe | Atkinson | MATKINSO@ORACLE.COM
131 | James | Marlow | JAMRLOW@ORACLE.COM
132 | TJ | Olson | TJOLSON@ORACLE.COM
133 | Jason | Mallin | JMALLIN@ORACLE.COM
134 | Michael | Rogers | MROGERS@ORACLE.COM
135 | Ki | Gee | KGEE@ORACLE.COM
136 | Hazel | Philtanker | HPHILTAN@ORACLE.COM
137 | Renske | Ladwig | RLADWIG@ORACLE.COM
138 | Stephen | Stiles | SSTILES@ORACLE.COM
139 | John | Seo | JSEO@ORACLE.COM
140 | Joshua | Patel | JPATEL@ORACLE.COM
141 | Trenna | Rajs | TRAJS@ORACLE.COM
142 | Curtis | Davies | CDAVIES@ORACLE.COM
143 | Randall | Matos | RMATOS@ORACLE.COM
144 | Peter | Vargas | PVARGAS@ORACLE.COM
145 | John | Russell | JRUSSEL@ORACLE.COM
146 | Karen | Partners | KPARTNER@ORACLE.COM
147 | Alberto | Errazuriz | AERRAZUR@ORACLE.COM
148 | Gerald | Cambrault | GCAMBRAU@ORACLE.COM
149 | Eleni | Zlotkey | EZLOTKEY@ORACLE.COM
150 | Peter | Tucker | PTUCKER@ORACLE.COM
151 | David | Bernstein | DBERNSTE@ORACLE.COM
152 | Peter | Hall | PHALL@ORACLE.COM
153 | Christopher | Olsen | COLSEN@ORACLE.COM
154 | Nanette | Cambrault | NCAMBRAU@ORACLE.COM
155 | Oliver | Tuvault | OTUVAULT@ORACLE.COM
156 | Janette | King | JKING@ORACLE.COM
157 | Patrick | Sully | PSULLY@ORACLE.COM
158 | Allan | McEwen | AMCEWEN@ORACLE.COM
159 | Lindsey | Smith | LSMITH@ORACLE.COM
160 | Louise | Doran | LDORAN@ORACLE.COM
161 | Sarath | Sewall | SSEWALL@ORACLE.COM
162 | Clara | Vishney | CVISHNEY@ORACLE.COM
163 | Danielle | Greene | DGREENE@ORACLE.COM
164 | Mattea | Marvins | MMARVINS@ORACLE.COM
165 | David | Lee | DLEE@ORACLE.COM
166 | Sundar | Ande | SANDE@ORACLE.COM
167 | Amit | Banda | ABANDA@ORACLE.COM
168 | Lisa | Ozer | LOZER@ORACLE.COM
169 | Harrison | Bloom | HBLOOM@ORACLE.COM
170 | Tayler | Fox | TFOX@ORACLE.COM
171 | William | Smith | WSMITH@ORACLE.COM
201 | Michael | Hartstein | MHARTSTE@ORACLE.COM
202 | Pat | Fay | PFAY@ORACLE.COM
203 | Susan | Mavris | SMAVRIS@ORACLE.COM
204 | Hermann | Baer | HBAER@ORACLE.COM
205 | Shelley | Higgins | SHIGGINS@ORACLE.COM
206 | William | Gietz | WGIETZ@ORACLE.COM
(107 rows)
hr#

Soal 2: membuat dan memanggil procedure restore_suffix_email



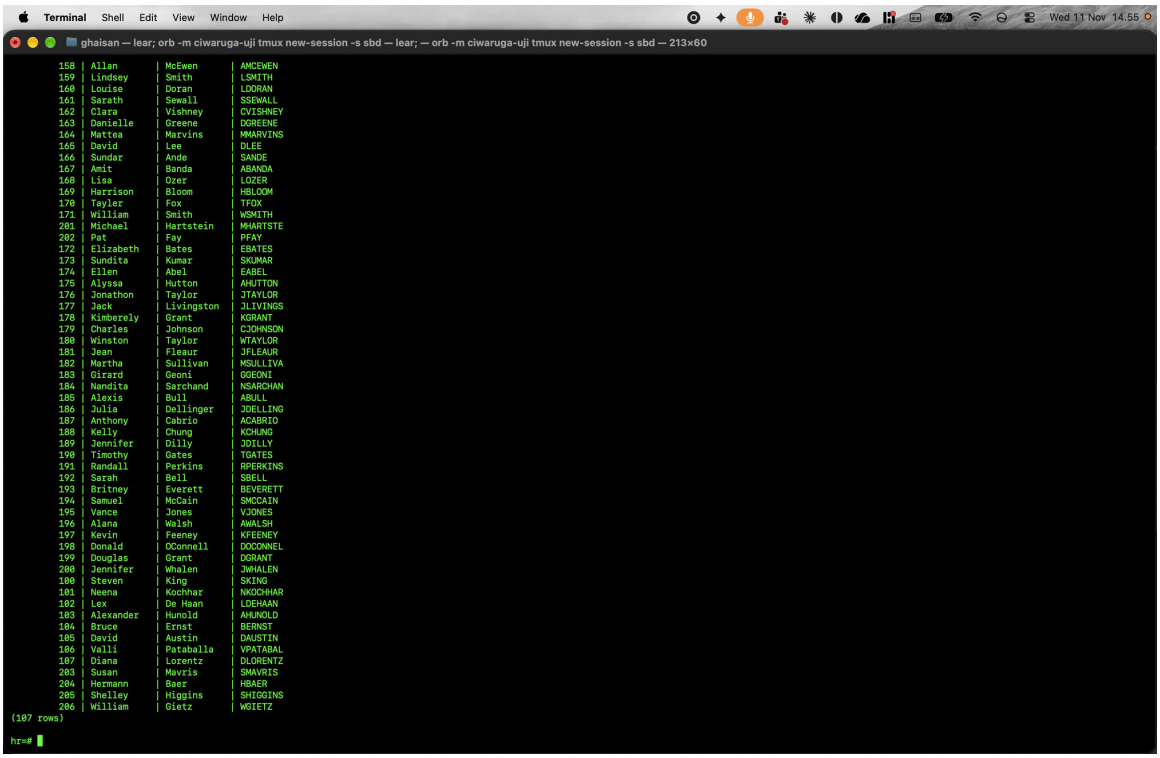
```
Terminal  Shell  Edit  View  Window  Help                                   Wed 11 Nov 14.55
ghaisan — lear; orb -m ciwaruga-uji tmux new-session -s sbd — lear; — orb -m ciwaruga-uji tmux new-session -s sbd — 213×60

hr=# CREATE OR REPLACE PROCEDURE restore_suffix_email()
hr-#   LANGUAGE plpgsql
hr-# AS $$
hr$# DECLARE
hr$#   jumlah INTEGER;
hr$# BEGIN
hr$#   UPDATE employees
hr$#   SET email = LEFT(email, LENGTH(email) - LENGTH('@ORACLE.COM'))
hr$#   WHERE email LIKE '%@ORACLE.COM';
hr$#
hr$#   GET DIAGNOSTICS jumlah = ROW_COUNT;
hr$#   RAISE NOTICE '% email dikembalikan ke semula', jumlah;
hr$# END;
hr$# $$;
CREATE PROCEDURE
hr=# CALL restore_suffix_email();
NOTICE:  107 email dikembalikan ke semula
CALL
hr=#
```

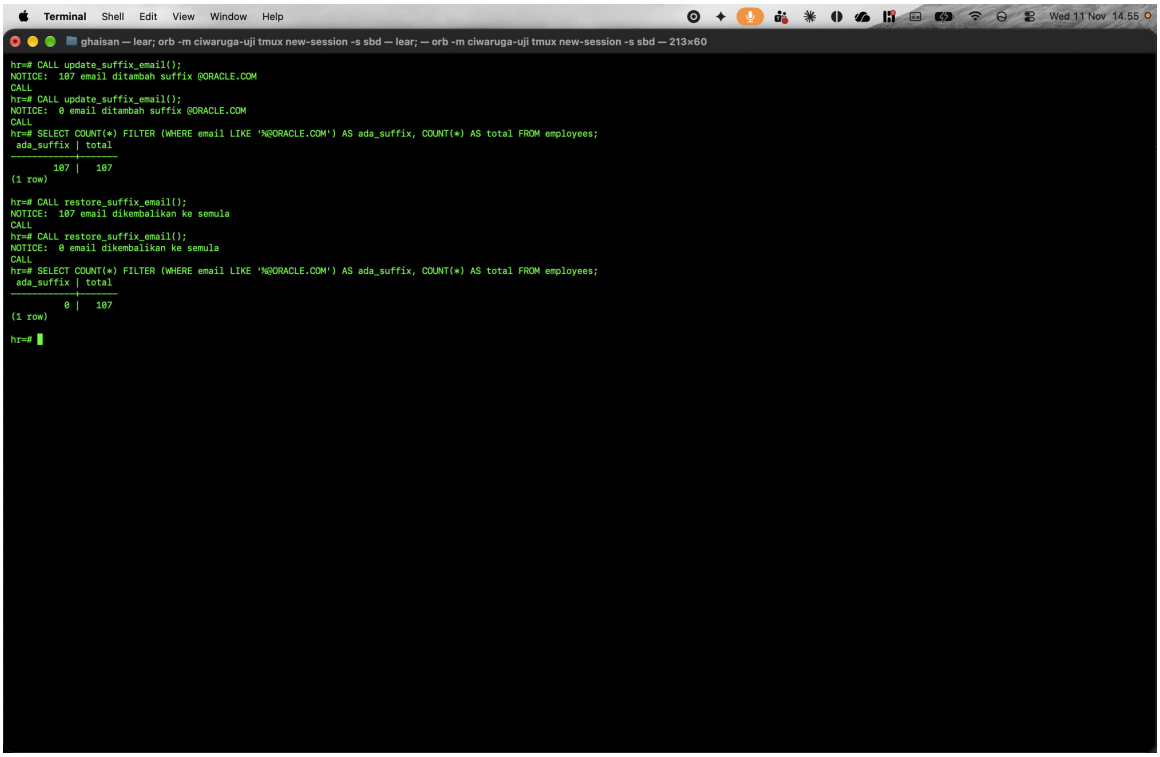


```
Terminal  Shell  Edit  View  Window  Help                                   Wed 11 Nov 14.55
ghaisan — lear; orb -m ciwaruga-uji tmux new-session -s sbd — lear; — orb -m ciwaruga-uji tmux new-session -s sbd — 213×60

hr=# SELECT employee_id, first_name, last_name, email FROM employees;                    14:55 [53/53]
 employee_id | first_name  |  last_name  |  email
-------------+-------------+-------------+----------
         100 | Nancy       | Greenberg   | NGREENBE
         109 | Daniel      | Faviet      | DFAVIET
         110 | John        | Chen        | JCHEN
         111 | Ismael      | Sciarra     | ISCIARRA
         112 | Jose Manuel | Urman       | JMURMAN
         113 | Luis        | Popp        | LPOPP
         114 | Den         | Raphaely    | DRAPHEAL
         115 | Alexander   | Khoo        | AKHOO
         116 | Shelli      | Baida       | SBAIDA
         117 | Sigal       | Tobias      | STOBIAS
         118 | Guy         | Himuro      | GHIMURO
         119 | Karen       | Colmenares  | KCOLMENA
         120 | Matthew     | Weiss       | MWEISS
         121 | Adam        | Fripp       | AFRIPP
         122 | Payam       | Kaufling    | PKAUFLIN
         123 | Shanta      | Vollman     | SVOLLMAN
         124 | Kevin       | Mourgos     | KMOURGOS
         125 | Julia       | Nayer       | JNAYER
         126 | Irene       | Mikkilineni | IMIKKILI
         127 | James       | Landry      | JLANDRY
         128 | Steven      | Markle      | SMARKLE
         129 | Laura       | Bissot      | LBISSOT
         130 | Mozhe       | Atkinson    | MATKINSO
         131 | James       | Marlow      | JAMRLOW
         132 | TJ          | Olson       | TJOLSON
         133 | Jason       | Mallin      | JMALLIN
         134 | Michael     | Rogers      | MROGERS
         135 | Ki          | Gee         | KGEE
         136 | Hazel       | Philtanker  | HPHILTAN
         137 | Renske      | Ladwig      | RLADWIG
         138 | Stephen     | Stiles      | SSTILES
         139 | John        | Seo         | JSEO
         140 | Joshua      | Patel       | JPATEL
         141 | Trenna      | Rajs        | TRAJS
         142 | Curtis      | Davies      | CDAVIES
         143 | Randall     | Matos       | RMATOS
         144 | Peter       | Vargas      | PVARGAS
         145 | John        | Russell     | JRUSSEL
         146 | Karen       | Partners    | KPARTNER
         147 | Alberto     | Errazuriz   | AERRAZUR
         148 | Gerald      | Cambrault   | GCAMBRAU
         149 | Eleni       | Zlotkey     | EZLOTKEY
         150 | Peter       | Tucker      | PTUCKER
         151 | David       | Bernstein   | DBERNSTE
         152 | Peter       | Hall        | PHALL
         153 | Christopher | Olsen       | COLSEN
         154 | Nanette     | Cambrault   | NCAMBRAU
         155 | Oliver      | Tuvault     | OTUVAULT
         156 | Janette     | King        | JKING
         157 | Patrick     | Sully       | PSULLY
         158 | Allan       | McEwen      | AMCEWEN
         159 | Lindsey     | Smith       | LSMITH
         160 | Louise      | Doran       | LDORAN
         161 | Sarath      | Sewall      | SSEWALL
         162 | Clara       | Vishney     | CVISHNEY
         163 | Danielle    | Greene      | DGREENE
         164 | Mattea      | Marvins     | MMARVINS
```



Soal 2 (percobaan tambahan): procedure dipanggil dua kali berturut-turut



Soal No. 3

Soal

Tuliskan perbedaan penggunaan Function & Procedure pada PL/pgSQL PostgreSQL minimal 5 hal!

Jawaban (Penjelasan dan Syntax)

Soal 3: perbedaan Function dan Procedure pada PL/pgSQL PostgreSQL

Procedure baru ada di PostgreSQL sejak versi 11, sebelumnya semua rutin ditulis sebagai function. Perbedaan keduanya:

No	Aspek	Function	Procedure
1	Cara memanggil	SELECT nama_function(...), bisa juga di dalam query lain	CALL nama_procedure(...) sebagai perintah tersendiri
2	Nilai kembali	Wajib RETURNS tipe tertentu (boleh VOID, TABLE, SETOF) dan memakai RETURN nilai	Tidak punya RETURNS, hasil dikirim lewat parameter OUT atau INOUT
3	Kontrol transaksi	Tidak boleh COMMIT/ROLLBACK, selalu ikut transaksi query pemanggil	Boleh COMMIT/ROLLBACK di dalam body kalau CALL tidak berada di dalam blok transaksi
4	Dipakai di dalam query	Bisa di SELECT, WHERE, GROUP BY, DEFAULT, CHECK, dan index (kalau IMMUTABLE)	Tidak bisa dipakai di dalam query
5	Dipakai trigger	Trigger menjalankan function yang RETURNS TRIGGER	Tidak bisa dipasang langsung di trigger
6	Perintah DDL dan atribut	CREATE/DROP FUNCTION, punya atribut IMMUTABLE/STABLE/VOLATILE, STRICT, PARALLEL	CREATE/DROP PROCEDURE, atribut-atribut itu tidak ada
7	Kegunaan umum	Menghitung atau mengolah data lalu mengembalikan nilai	Menjalankan proses atau aksi, misalnya batch UPDATE

Perbedaan nomor 1, 3, 4, dan 6 dibuktikan langsung pada percobaan di bawah. Nomor 2 terlihat di soal 5b (procedure dengan parameter INOUT), dan nomor 5 dibahas di tugas P10.2 Trigger.

Soal 3 (percobaan): cara memanggil function dan procedure

Percobaan untuk perbedaan nomor 1 dan 4. Function dipanggil dengan SELECT dan hasilnya bisa dipakai di dalam query, termasuk di WHERE (query terakhir menampilkan 6 employee grade S).

Kalau function dipanggil dengan CALL, PostgreSQL menolak dengan pesan salary_grade(integer) is not a procedure dan petunjuk To call a function, use SELECT. Sebaliknya procedure yang dipanggil dengan SELECT ditolak dengan pesan update_suffix_email() is a procedure dan petunjuk To call a procedure, use CALL.

```
SELECT salary_grade(15000) AS grade;

CALL salary_grade(15000);

SELECT update_suffix_email();

SELECT employee_id, salary_grade(salary) FROM employees WHERE salary_grade(salary) =
    'S';
```

Soal 3 (percobaan): COMMIT di dalam procedure dan di dalam function

Percobaan untuk perbedaan nomor 3. Procedure coba_commit berhasil menjalankan COMMIT di tengah body: kedua NOTICE muncul dan CALL selesai tanpa error. Ini bisa karena CALL dijalankan langsung, tidak di dalam blok BEGIN ... COMMIT.

Function coba_commit_fn yang berisi COMMIT memang bisa dibuat, tetapi saat dipanggil keluar ERROR: invalid transaction termination. Function selalu berjalan di dalam transaksi milik query yang memanggilnya, jadi tidak boleh mengakhiri transaksi itu.

```
CREATE OR REPLACE PROCEDURE coba_commit()
  LANGUAGE plpgsql
AS $$
BEGIN
  RAISE NOTICE 'procedure: sebelum COMMIT';
  COMMIT;
  RAISE NOTICE 'procedure: sesudah COMMIT, transaksi baru dimulai otomatis';
END;
$$;

CALL coba_commit();

CREATE OR REPLACE FUNCTION coba_commit_fn()
  RETURNS VOID
  LANGUAGE plpgsql
AS $$
BEGIN
  COMMIT;
END;
$$;

SELECT coba_commit_fn();
```

Soal 3 (percobaan): DROP FUNCTION dan DROP PROCEDURE

Percobaan untuk perbedaan nomor 6. Function dan procedure dihapus dengan perintah berbeda, DROP FUNCTION dan DROP PROCEDURE. Daftar parameternya ikut ditulis karena nama yang sama boleh dipakai untuk parameter yang berbeda (overloading).

Di katalog pg_proc keduanya tersimpan di tabel yang sama, bedanya di kolom prokind: f untuk function dan p untuk procedure. Setelah di-drop, coba_commit dan coba_commit_fn tidak ada lagi di hasil query.

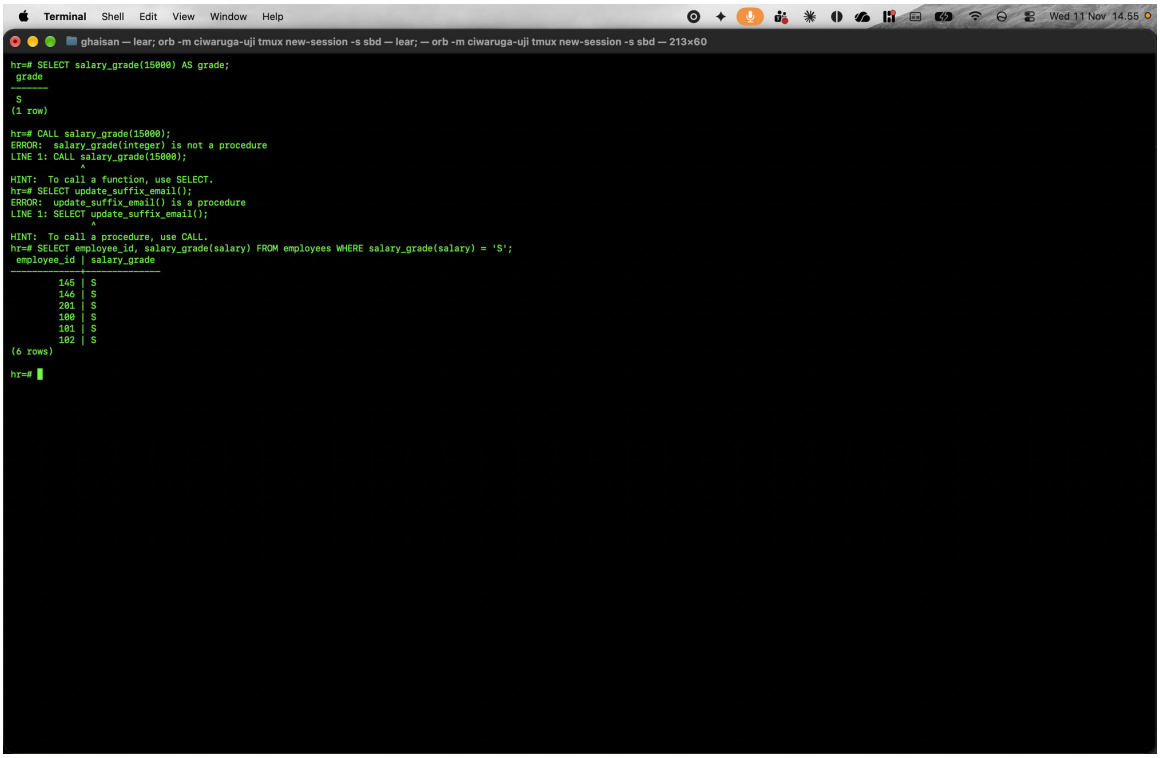
```
DROP FUNCTION coba_commit_fn();

DROP PROCEDURE coba_commit();

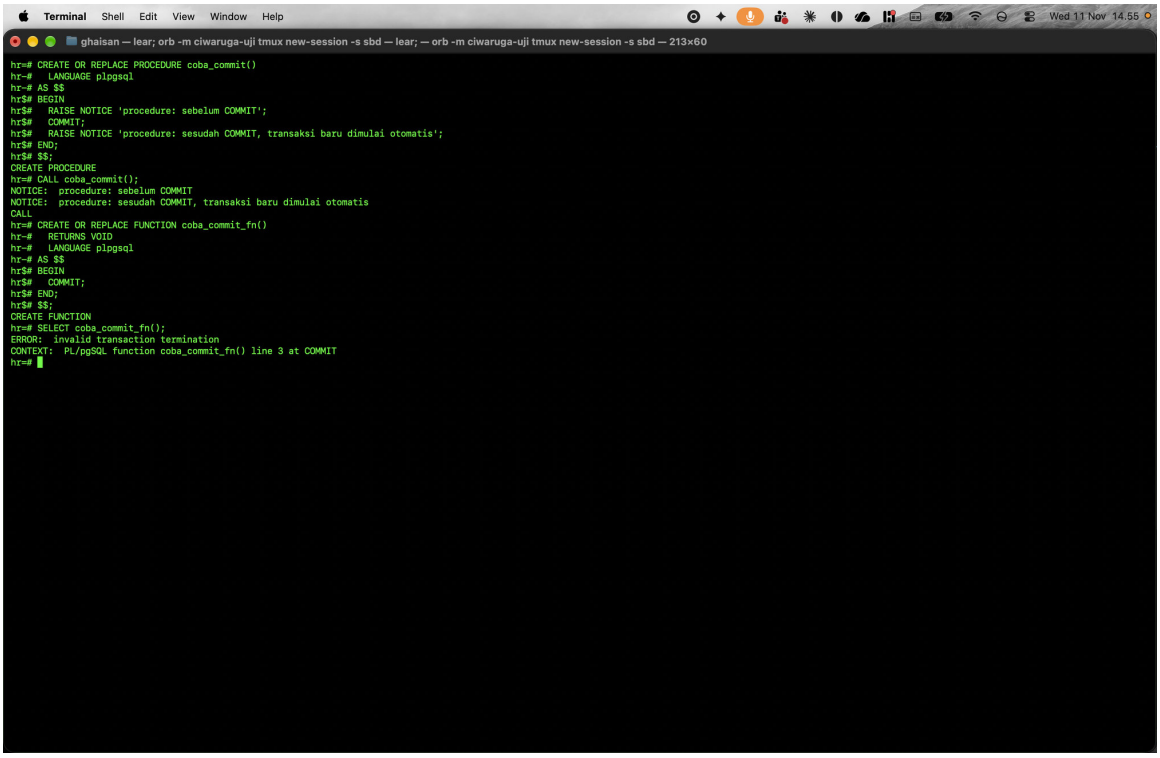
SELECT proname, prokind FROM pg_proc WHERE proname IN ('salary_grade',
    'update_suffix_email', 'restore_suffix_email', 'coba_commit', 'coba_commit_fn')
    ORDER BY proname;
```

Hasil Eksekusi (Screenshot)

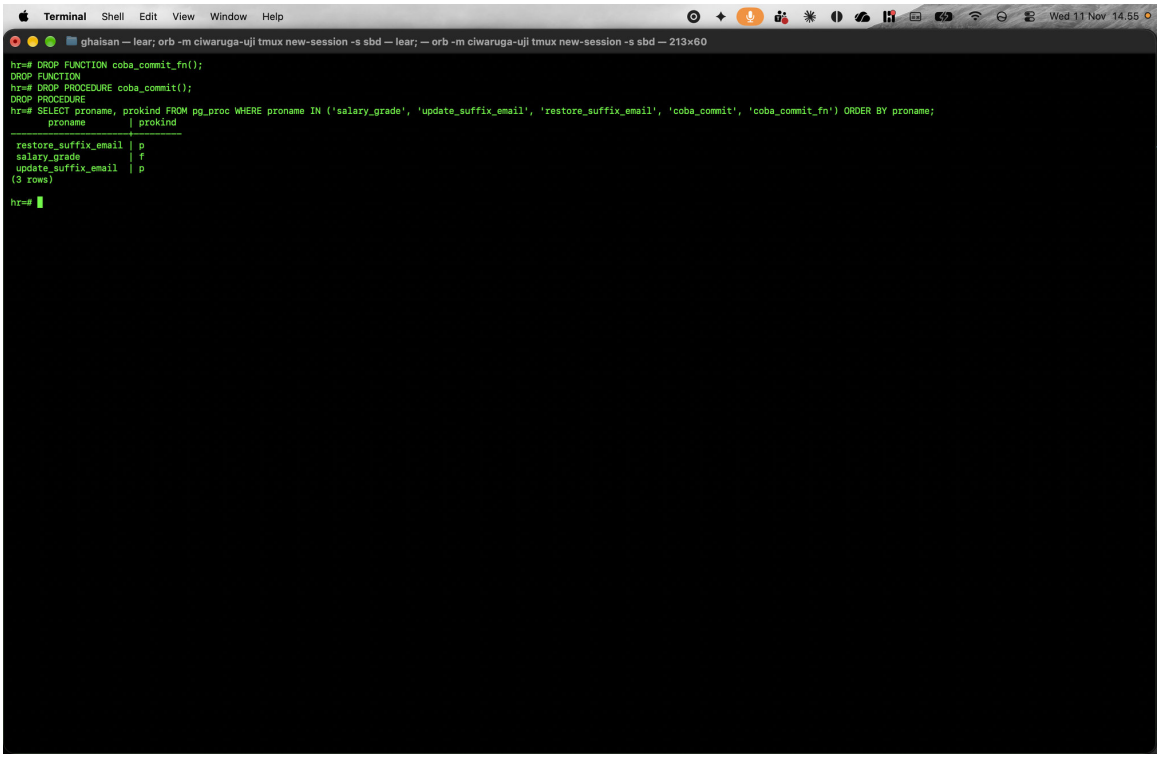
Soal 3 (percobaan): cara memanggil function dan procedure



Soal 3 (percobaan): COMMIT di dalam procedure dan di dalam function



Soal 3 (percobaan): DROP FUNCTION dan DROP PROCEDURE



Soal No. 4

Soal
a. Buatlah contoh suatu procedure yang di dalamnya memanggil procedure! Contoh tersebut harus melibatkan salah satu tabel pada skema HR! b. Buatlah contoh suatu function yang di dalamnya memanggil function! Contoh tersebut harus melibatkan salah satu tabel pada skema HR!
Jawaban (Penjelasan dan Syntax)

Soal 4a: procedure naikkan_gaji_department yang memanggil procedure naikkan_gaji_employee

Procedure naikkan_gaji_employee menaikkan gaji satu employee sebesar p_persen persen. Rumusnya salary * (1 + p_persen / 100), dibulatkan 2 angka di belakang koma sesuai tipe numeric(8,2).

Procedure naikkan_gaji_department mengambil semua employee di satu department dengan FOR ... IN SELECT ... LOOP, lalu untuk setiap baris memanggil procedure naikkan_gaji_employee lewat CALL. Jadi di dalam procedure, procedure lain dipanggil dengan cara yang sama seperti dari psql, yaitu CALL.

Tabel HR yang dipakai: employees.

```
CREATE OR REPLACE PROCEDURE naikkan_gaji_employee(p_employee_id INTEGER, p_persen
    NUMERIC)
  LANGUAGE plpgsql
AS $$
BEGIN
  UPDATE employees
  SET salary = ROUND(salary * (1 + p_persen / 100), 2)
  WHERE employee_id = p_employee_id;
END;
$$;
```

```
CREATE OR REPLACE PROCEDURE naikkan_gaji_department(p_department_id INTEGER, p_persen
    NUMERIC)
  LANGUAGE plpgsql
AS $$
DECLARE
  emp RECORD;
BEGIN
  FOR emp IN SELECT employee_id, first_name, salary FROM employees
             WHERE department_id = p_department_id ORDER BY employee_id LOOP
    -- procedure memanggil procedure lain untuk tiap employee
    CALL naikkan_gaji_employee(emp.employee_id, p_persen);
    RAISE NOTICE 'Gaji % (ID %) naik % persen dari %',
      emp.first_name, emp.employee_id, p_persen, emp.salary;
  END LOOP;
END;
$$;
```

Soal 4a: pengujian di dalam transaksi lalu di-ROLLBACK supaya data HR tetap asli

Gaji department 60 (IT) sebelum procedure dijalankan: Alexander 9000, Bruce 6000, David 4800, Valli 4800, dan Diana 4200.

Supaya data HR tidak berubah permanen, pengujian dilakukan di dalam transaksi: BEGIN, CALL naikkan_gaji_department(60, 10), cek hasil, lalu ROLLBACK. Setelah CALL muncul 5 NOTICE dari procedure luar, dan gaji kelima employee naik 10 persen menjadi 9900, 6600, 5280, 5280, dan 4620.

Setelah ROLLBACK, query yang sama menampilkan gaji kembali ke nilai awal. Prompt psql berubah menjadi hr=*# selama transaksi masih terbuka.

```
SELECT employee_id, first_name, salary FROM employees WHERE department_id = 60 ORDER
    BY employee_id;

BEGIN;

CALL naikkan_gaji_department(60, 10);

SELECT employee_id, first_name, salary FROM employees WHERE department_id = 60 ORDER
    BY employee_id;

ROLLBACK;

SELECT employee_id, first_name, salary FROM employees WHERE department_id = 60 ORDER
    BY employee_id;
```

Soal 4b: function profil_employee yang memanggil function nama_lengkap dan salary_grade

Function nama_lengkap mengembalikan first_name dan last_name yang digabung dengan spasi berdasarkan employee_id.

Function profil_employee mengambil salary employee, lalu di dalam RETURN memanggil dua function lain, nama_lengkap() dan salary_grade() dari soal 1. Function dipanggil dari function lain cukup dengan menuliskannya di ekspresi, sama seperti di SELECT.

IF NOT FOUND dipakai untuk employee_id yang tidak ada. FOUND bernilai false kalau SELECT INTO tidak menemukan baris, jadi profil_employee(999) mengembalikan Employee 999 tidak ditemukan.

Hasil untuk department 90: Steven King, salary 24000.00, grade S; Neena Kochhar, salary 17000.00, grade S; dan Lex De Haan, salary 17000.00, grade S.

```
CREATE OR REPLACE FUNCTION nama_lengkap(p_employee_id INTEGER)
  RETURNS VARCHAR
  LANGUAGE plpgsql
AS $$
DECLARE
  hasil VARCHAR;
BEGIN
  SELECT first_name || ' ' || last_name INTO hasil
  FROM employees
  WHERE employee_id = p_employee_id;
  RETURN hasil;
```

```
END;
$$;

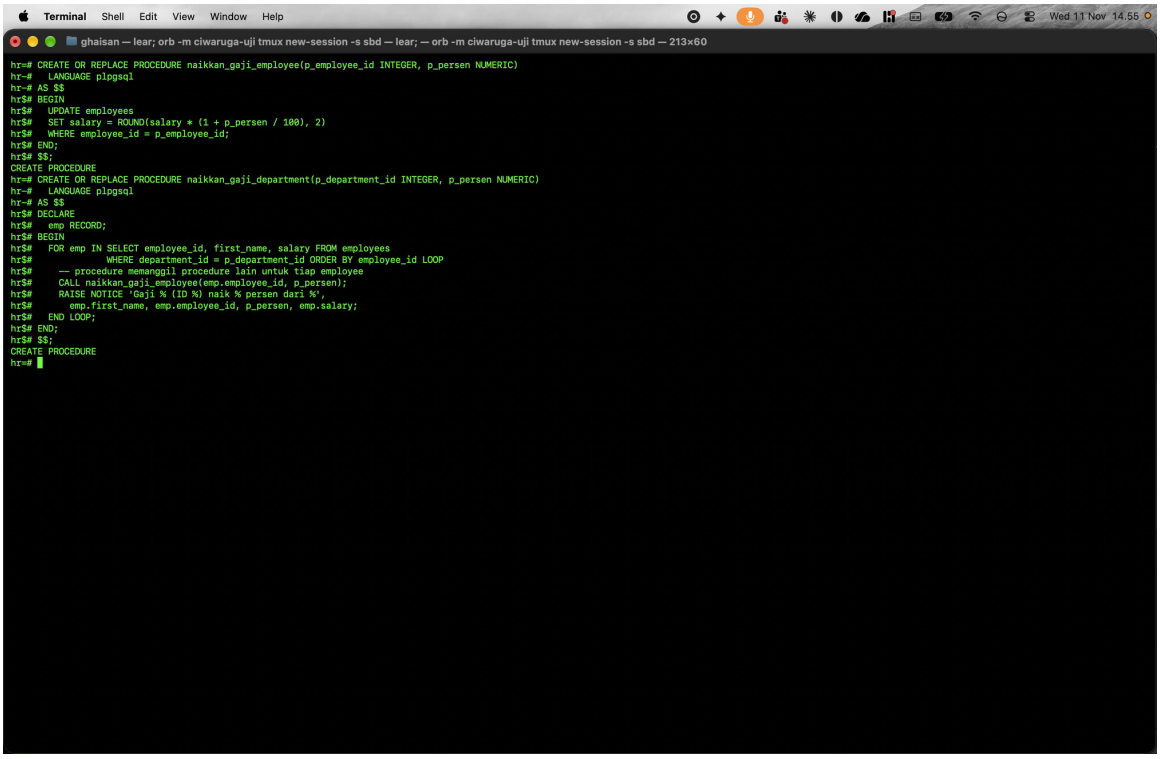
CREATE OR REPLACE FUNCTION profil_employee(p_employee_id INTEGER)
  RETURNS VARCHAR
  LANGUAGE plpgsql
AS $$
DECLARE
  v_salary employees.salary%TYPE;
BEGIN
  SELECT salary INTO v_salary FROM employees WHERE employee_id = p_employee_id;
  IF NOT FOUND THEN
    RETURN 'Employee ' || p_employee_id || ' tidak ditemukan';
  END IF;
  -- function memanggil function nama_lengkap() dan salary_grade()
  RETURN nama_lengkap(p_employee_id) || ', salary ' || v_salary
         || ', grade ' || salary_grade(v_salary);
END;
$$;

SELECT employee_id, profil_employee(employee_id) FROM employees WHERE department_id =
    90;

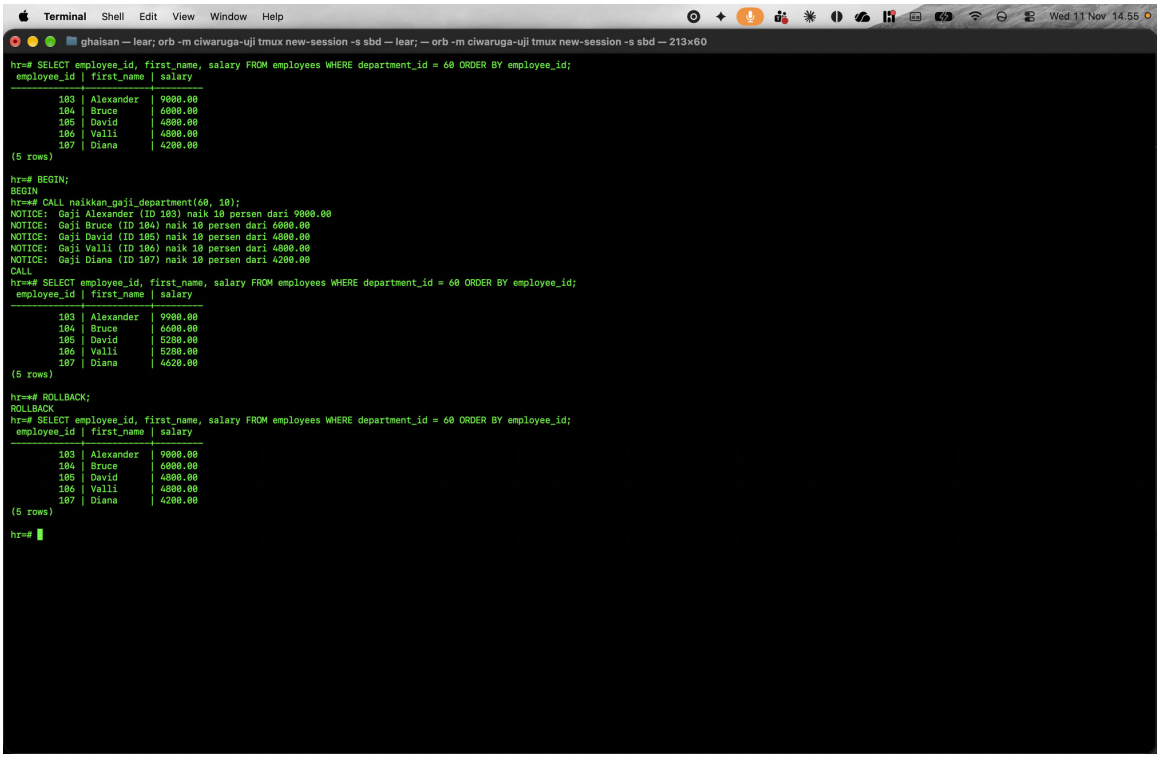
SELECT profil_employee(999);
```

Hasil Eksekusi (Screenshot)

Soal 4a: procedure naikkan_gaji_department yang memanggil procedure naikkan_gaji_employee



Soal 4a: pengujian di dalam transaksi lalu di-ROLLBACK supaya data HR tetap asli



```
Terminal  Shell  Edit  View  Window  Help                                  Wed 11 Nov 14.55
ghaisan — lear; orb -m ciwaruga-uji tmux new-session -s sbd — lear; — orb -m ciwaruga-uji tmux new-session -s sbd — 213×60

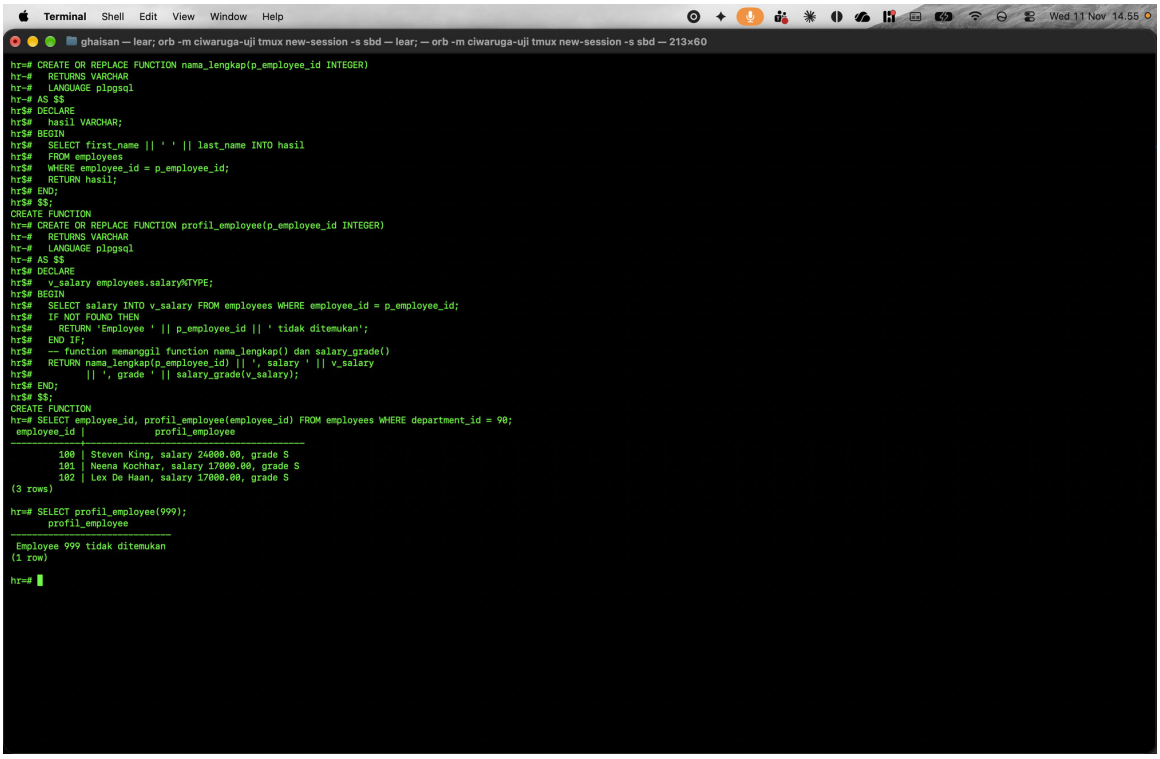
hr=# SELECT employee_id, first_name, salary FROM employees WHERE department_id = 60 ORDER BY employee_id;
 employee_id | first_name | salary
-------------+------------+---------
         103 | Alexander  | 9000.00
         104 | Bruce      | 6000.00
         105 | David      | 4800.00
         106 | Valli      | 4800.00
         107 | Diana      | 4200.00
(5 rows)

hr=# BEGIN;
BEGIN
hr=*# CALL naikkan_gaji_department(60, 10);
NOTICE:  Gaji Alexander (ID 103) naik 10 persen dari 9000.00
NOTICE:  Gaji Bruce (ID 104) naik 10 persen dari 6000.00
NOTICE:  Gaji David (ID 105) naik 10 persen dari 4800.00
NOTICE:  Gaji Valli (ID 106) naik 10 persen dari 4800.00
NOTICE:  Gaji Diana (ID 107) naik 10 persen dari 4200.00
CALL
hr=*# SELECT employee_id, first_name, salary FROM employees WHERE department_id = 60 ORDER BY employee_id;
 employee_id | first_name | salary
-------------+------------+---------
         103 | Alexander  | 9900.00
         104 | Bruce      | 6600.00
         105 | David      | 5280.00
         106 | Valli      | 5280.00
         107 | Diana      | 4620.00
(5 rows)

hr=*# ROLLBACK;
ROLLBACK
hr=# SELECT employee_id, first_name, salary FROM employees WHERE department_id = 60 ORDER BY employee_id;
 employee_id | first_name | salary
-------------+------------+---------
         103 | Alexander  | 9000.00
         104 | Bruce      | 6000.00
         105 | David      | 4800.00
         106 | Valli      | 4800.00
         107 | Diana      | 4200.00
(5 rows)

hr=#
```

Soal 4b: function profil_employee yang memanggil function nama_lengkap dan salary_grade



```
Terminal  Shell  Edit  View  Window  Help                                  Wed 11 Nov 14.55
ghaisan — lear; orb -m ciwaruga-uji tmux new-session -s sbd — lear; — orb -m ciwaruga-uji tmux new-session -s sbd — 213×60

hr=# CREATE OR REPLACE FUNCTION nama_lengkap(p_employee_id INTEGER)
hr-#   RETURNS VARCHAR
hr-#   LANGUAGE plpgsql
hr-# AS $$
hr$# DECLARE
hr$#   hasil VARCHAR;
hr$# BEGIN
hr$#   SELECT first_name || ' ' || last_name INTO hasil
hr$#   FROM employees
hr$#   WHERE employee_id = p_employee_id;
hr$#   RETURN hasil;
hr$# END;
hr$# $$;
CREATE FUNCTION
hr=# CREATE OR REPLACE FUNCTION profil_employee(p_employee_id INTEGER)
hr-#   RETURNS VARCHAR
hr-#   LANGUAGE plpgsql
hr-# AS $$
hr$# DECLARE
hr$#   v_salary employees.salary%TYPE;
hr$# BEGIN
hr$#   SELECT salary INTO v_salary FROM employees WHERE employee_id = p_employee_id;
hr$#   IF NOT FOUND THEN
hr$#     RETURN 'Employee ' || p_employee_id || ' tidak ditemukan';
hr$#   END IF;
hr$#   -- function memanggil function nama_lengkap() dan salary_grade()
hr$#   RETURN nama_lengkap(p_employee_id) || ', salary ' || v_salary
hr$#          || ', grade ' || salary_grade(v_salary);
hr$# END;
hr$# $$;
CREATE FUNCTION
hr=# SELECT employee_id, profil_employee(employee_id) FROM employees WHERE department_id = 90;
 employee_id |                profil_employee
-------------+-----------------------------------------------
         100 | Steven King, salary 24000.00, grade S
         101 | Neena Kochhar, salary 17000.00, grade S
         102 | Lex De Haan, salary 17000.00, grade S
(3 rows)

hr=# SELECT profil_employee(999);
       profil_employee
-----------------------------
 Employee 999 tidak ditemukan
(1 row)

hr=#
```

Soal No. 5

Soal

a. Buatlah contoh suatu procedure yang di dalamnya memanggil function! Contoh tersebut harus melibatkan salah satu tabel pada skema HR!

b. Buatlah contoh suatu function yang di dalamnya memanggil procedure! Contoh tersebut harus melibatkan salah satu tabel pada skema HR!

Jawaban (Penjelasan dan Syntax)

Soal 5a: procedure cetak_grade_department yang memanggil function nama_lengkap dan salary_grade

Procedure cetak_grade_department mengulang semua employee di satu department (urut dari gaji terbesar) dan di setiap baris memanggil function nama_lengkap() dan salary_grade() di dalam RAISE NOTICE.

Procedure tidak bisa mengembalikan tabel seperti SELECT, jadi hasilnya ditampilkan lewat NOTICE. CALL cetak_grade_department(100) menampilkan 6 employee department Finance, dari Nancy Greenberg (12000, grade A) sampai Luis Popp (6900, grade C).

```
CREATE OR REPLACE PROCEDURE cetak_grade_department(p_department_id INTEGER)
  LANGUAGE plpgsql
AS $$
DECLARE
  emp RECORD;
BEGIN
  FOR emp IN SELECT employee_id, salary FROM employees
             WHERE department_id = p_department_id ORDER BY salary DESC LOOP
    -- procedure memanggil function nama_lengkap() dan salary_grade()
    RAISE NOTICE '% (ID %): salary %, grade %',
      nama_lengkap(emp.employee_id), emp.employee_id, emp.salary,
    salary_grade(emp.salary);
  END LOOP;
END;
$$;

CALL cetak_grade_department(100);
```

Soal 5b: function rata_gaji_department yang memanggil procedure hitung_gaji_department

Procedure hitung_gaji_department menghitung jumlah employee dan total gaji satu department. Procedure tidak punya nilai kembali, jadi hasilnya dikirim lewat dua parameter INOUT, p_jumlah dan p_total. DEFAULT NULL dipakai supaya procedure bisa dipanggil hanya dengan id department.

Function rata_gaji_department memanggil procedure itu dengan CALL hitung_gaji_department(p_department_id, jumlah, total). Variabel jumlah dan total diisi oleh procedure, lalu function mengembalikan total dibagi jumlah. Kalau department tidak punya employee, function mengembalikan 0 supaya tidak terjadi pembagian dengan nol.

CALL hitung_gaji_department(60) dari psql menampilkan nilai parameter INOUT sebagai satu baris: 5 employee dengan total gaji 28800.00.

Function boleh memanggil procedure selama procedure itu tidak menjalankan COMMIT atau ROLLBACK, karena function selalu berada di dalam transaksi pemanggilnya (lihat percobaan COMMIT di soal 3).

```
CREATE OR REPLACE PROCEDURE hitung_gaji_department(
  p_department_id INTEGER,
  INOUT p_jumlah INTEGER DEFAULT NULL,
  INOUT p_total NUMERIC DEFAULT NULL)
  LANGUAGE plpgsql
AS $$
BEGIN
  SELECT COUNT(*), COALESCE(SUM(salary), 0) INTO p_jumlah, p_total
  FROM employees
  WHERE department_id = p_department_id;
END;
```

```
$$;

CREATE OR REPLACE FUNCTION rata_gaji_department(p_department_id INTEGER)
  RETURNS NUMERIC
  LANGUAGE plpgsql
AS $$
DECLARE
  jumlah INTEGER;
  total NUMERIC;
BEGIN
  -- function memanggil procedure, hasilnya diterima lewat parameter INOUT
  CALL hitung_gaji_department(p_department_id, jumlah, total);
  IF jumlah = 0 THEN
    RETURN 0;
  END IF;
  RETURN ROUND(total / jumlah, 2);
END;
$$;

CALL hitung_gaji_department(60);
```

Soal 5b: pengujian function rata_gaji_department dan pembanding dengan AVG

Function dipakai di SELECT untuk 7 department. Hasilnya sama dengan pembanding ROUND(AVG(salary), 2), misalnya department 50 (Shipping) 3475.56 dan department 90 (Executive) 19333.33.

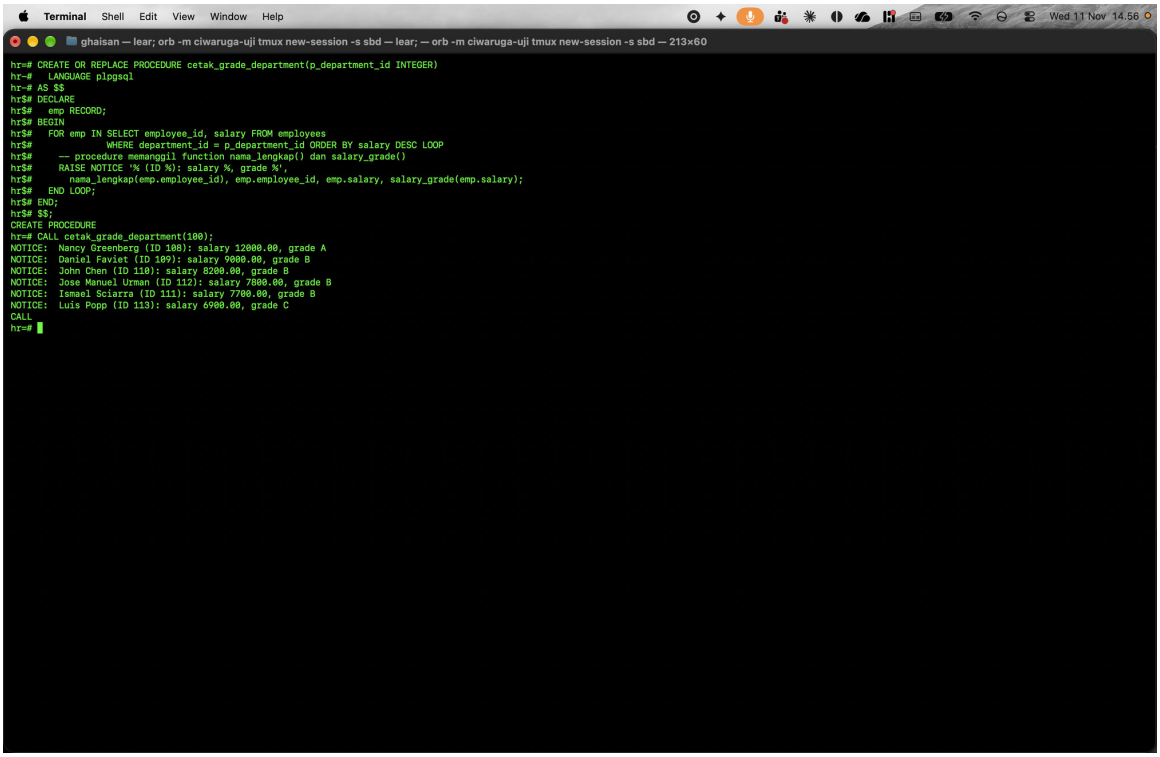
Department 120 (Treasury) tidak punya employee, jadi rata_gaji_department mengembalikan 0, sedangkan query AVG tidak menampilkan baris untuk department itu.

```
SELECT department_id, department_name, rata_gaji_department(department_id) AS
    rata_gaji
FROM departments
WHERE department_id IN (10, 20, 50, 60, 90, 100, 120)
ORDER BY department_id;

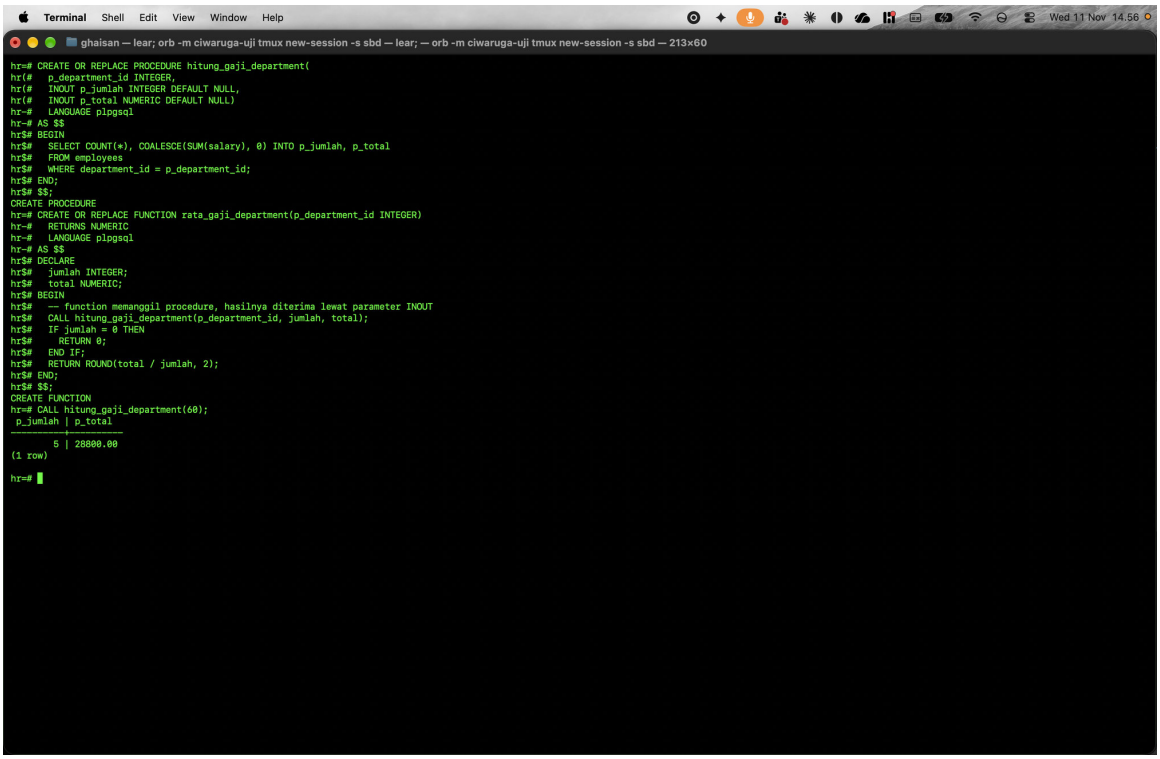
SELECT department_id, ROUND(AVG(salary), 2) AS avg_salary
FROM employees
WHERE department_id IN (10, 20, 50, 60, 90, 100, 120)
GROUP BY department_id
ORDER BY department_id;
```

Hasil Eksekusi (Screenshot)

Soal 5a: procedure cetak_grade_department yang memanggil function nama_lengkap dan salary_grade



Soal 5b: function rata_gaji_department yang memanggil procedure hitung_gaji_department



Soal 5b: pengujian function rata_gaji_department dan pembanding dengan AVG

